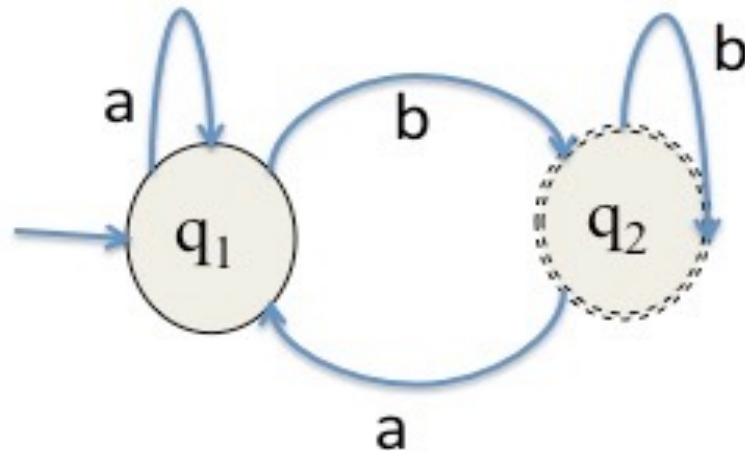


Automi finiti deterministici

In DFA, lo stato successivo occupato in corrispondenza di un dato input è unicamente determinato



Quindi le macchine sono **deterministiche**.

La funzione di transizione in un DFA è $f : Q \times \Sigma \rightarrow Q$.

Restituisce sempre un singolo stato

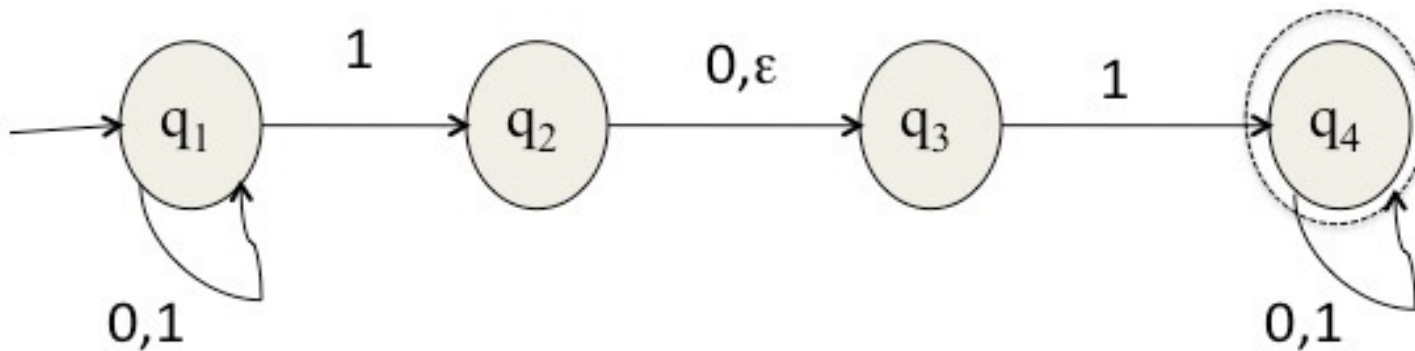
Automi finiti non deterministici

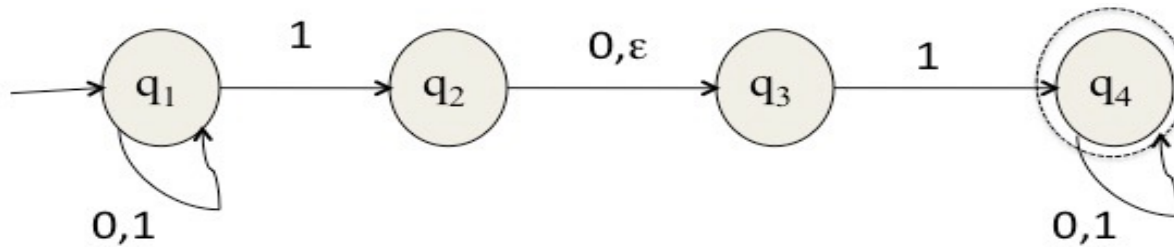
Un automa finito non deterministico (NFA)

Per uno stato q può di avere

- 0, 1 o anche più archi uscenti per ciascun simbolo dell'alfabeto
- archi uscenti con etichetta ϵ (epsilon-transizioni)

Esempio: NFA N1 con alfabeto $\Sigma = \{0,1\}$





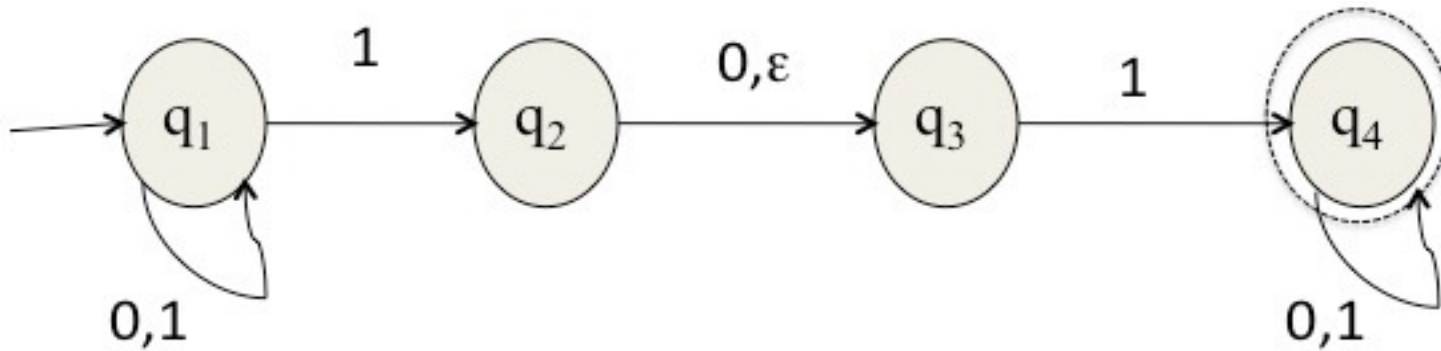
Se un NFA giunge in uno stato e, leggendo un simbolo a dell'alfabeto, si trova davanti a più scelte (più archi uscenti etichettati con a):

- la macchina si divide in **più copie** di se stessa, una copia per ciascuna scelta;
- ogni copia continua la computazione in parallelo indipendentemente dalle altre

Se una copia di un NFA legge un simbolo che non si trova su alcun arco uscente da quello stato, allora quella copia della macchina termina la computazione (senza risultato)

Accettazione

- Se almeno una copia giunge in uno stato finale, la stringa è accettata.
- Se nessuna copia giunge in uno stato finale, allora la stringa non è accettata.



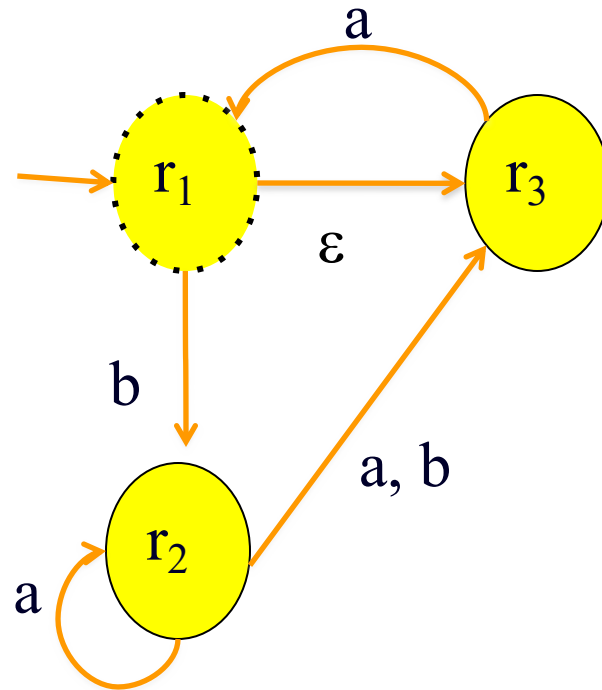
Se in stato con ε -transition, senza leggere alcun simbolo input,
NFA si divide in più copie,

- una che segue ciascun arco etichettato con ϵ
- una che resta nello stato corrente

ogni copia continua indipendentemente dalle altre

NFA continua **non deterministicamente** come prima.

Su input 10110 ?



NFA N accetta stringhe $\varepsilon, a, aa, baa, baba, \dots$

NFA N non accetta stringhe $b, ba, bb, \dots$

Definizione di NFA

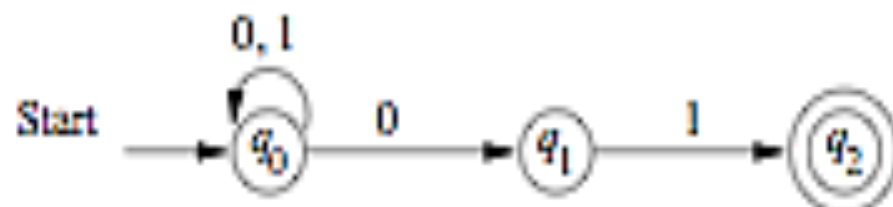
Def.: Per alfabeto Σ , sia Σ_ε ottenuto da Σ aggiungendo ε

Def.: Un **NFA** è 5-tupla (Q, Σ, f, q_0, F) , dove

1. Q è l'insieme degli stati
2. Σ è l'alfabeto
3. $f : Q \times \Sigma_\varepsilon \rightarrow P(Q)$ è la funzione di transizione
4. $q_0 \in Q$ è lo stato iniziale
5. F è l'insieme degli stati finali

Nota: La differenza tra DFA e NFA è nella funzione di transizione f :

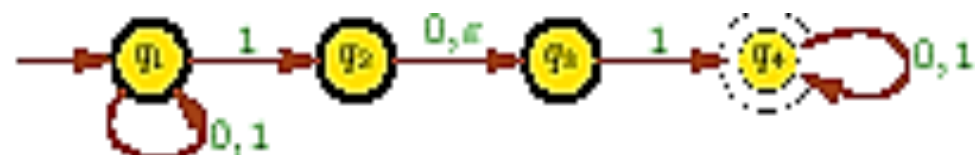
- ammette mosse di tipo ε
- Restituisce un' insieme di stati invece di un singolo stato.



$$(\{q_0, q_1, q_2\}, \{0, 1\}, \delta, q_0, \{q_2\})$$

dove δ e' la funzione di transizione

	0	1
$\rightarrow q_0$	$\{q_0, q_1\}$	$\{q_0\}$
q_1	$\emptyset$	$\{q_2\}$
$*q_2$	$\emptyset$	$\emptyset$



Formal description of above NFA $N = (Q, \Sigma, \delta, q_1, F)$

- $Q = \{q_1, q_2, q_3, q_4\}$
- $\Sigma = \{0, 1\}$
- Transition function $\delta : Q \times \Sigma_{\epsilon} \rightarrow \mathcal{P}(Q)$

	0	1	ϵ
q_1	$\{q_1\}$	$\{q_1, q_2\}$	$\emptyset$
q_2	$\{q_3\}$	$\emptyset$	$\{q_3\}$
q_3	$\emptyset$	$\{q_4\}$	$\emptyset$
q_4	$\{q_4\}$	$\{q_4\}$	$\emptyset$

- q_1 is the start state
- $F = \{q_4\}$

Computazione di NFA

Siano $N = (Q, \Sigma, f, q_0, F)$ un NFA e w una stringa su Σ

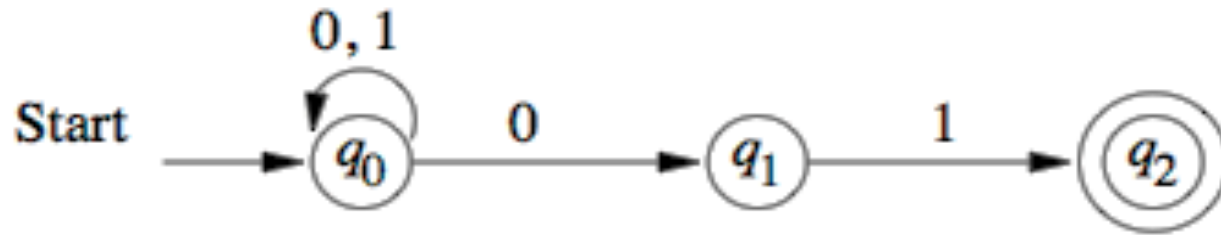
N **accetta** w se esiste almeno una computazione accettante:
 w può essere scritta come $y_1 y_2 \cdots y_m$, dove ogni y_i in Σ^* , e
esiste sequenza di stati $r_0, r_1, \dots, r_m$ in Q t.c.

1. $r_0 = q_0$
2. r_{i+1} in $f(r_i, y_{i+1})$ per ogni $i = 0, 1, 2, \dots, m - 1$
3. r_m in F

Def.: insieme di stringhe accettate da NFA N è
il linguaggio **riconosciuto** da N ed è denotato con $L(N)$.

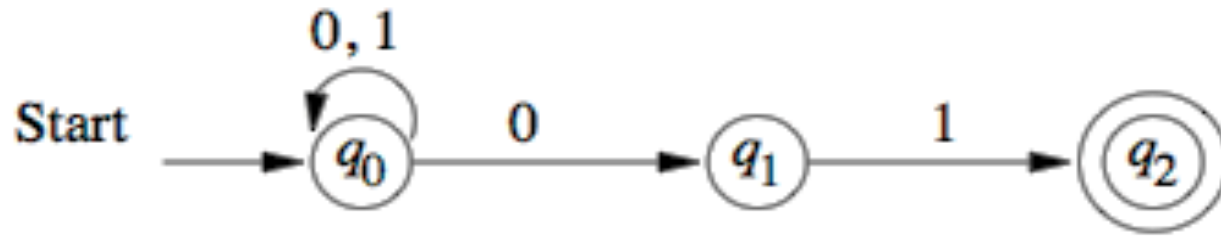
Nota: ogni DFA è anche un NFA.

Es.



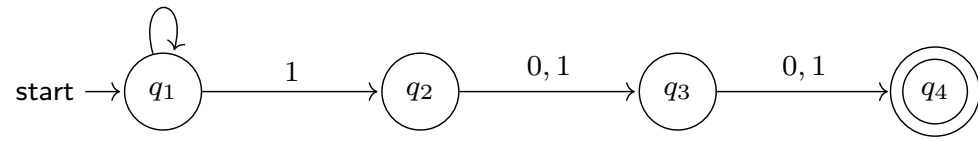
accetta il linguaggio ...

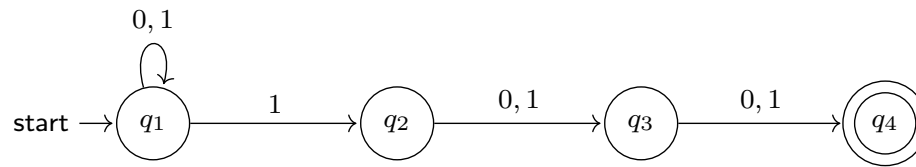
Es.



accetta il linguaggio $\{x01 : x \in \Sigma^*\}$.

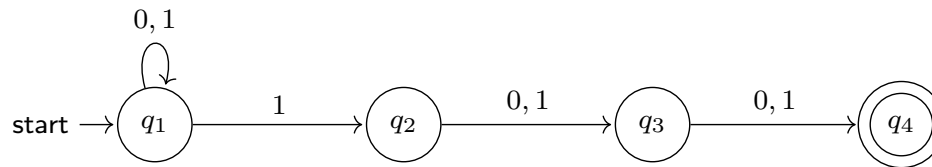
Input 01001?





riconosce il linguaggio di tutte le stringhe su $\{0, 1\}$ aventi un 1 nella terzultima posizione.

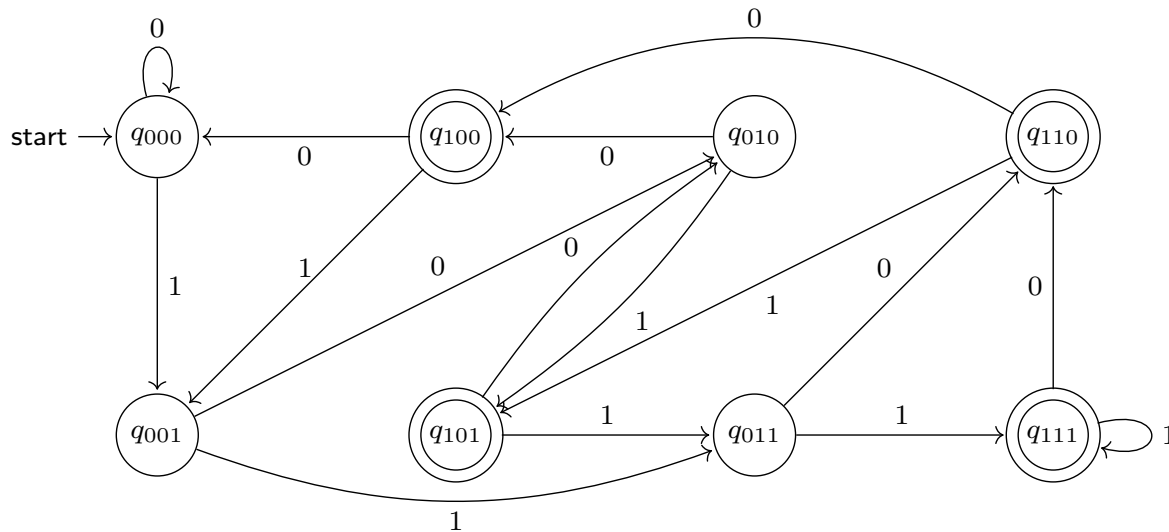
Durante la lettura della stringa input, una macchina sta sempre in q_1 e un'altra macchina si attiva ad ogni 1 per verificare se la stringa termina dopo 2 simboli.



riconosce il linguaggio di tutte le stringhe su $\{0, 1\}$ aventi un 1 nella terzultima posizione.

Durante la lettura della stringa input, una macchina sta sempre in q_1 e un'altra macchina si attiva ad ogni 1 per verificare se la stringa termina dopo 2 simboli.

Più piccolo DFA che riconosce lo stesso linguaggio (8 stati).



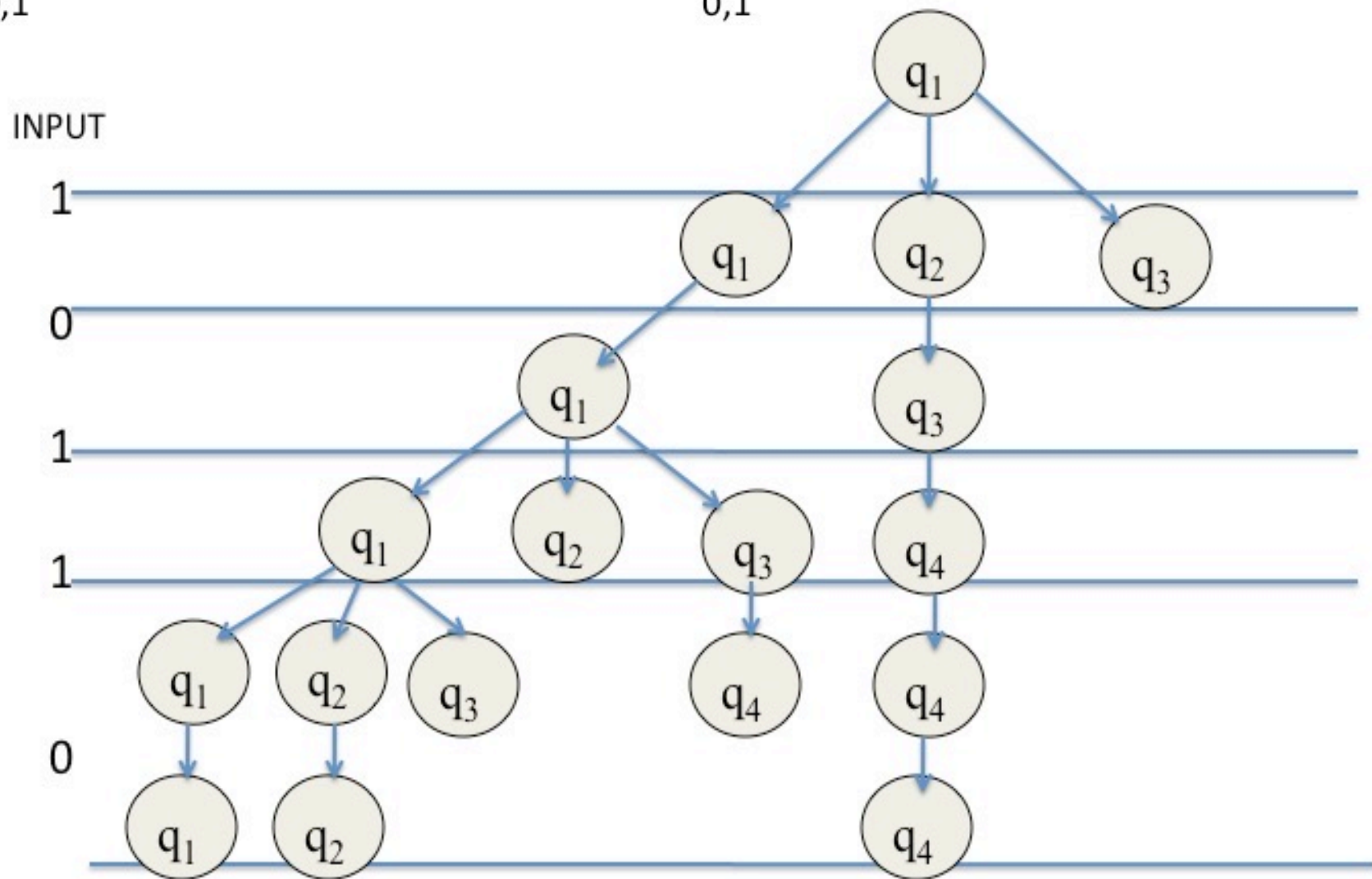
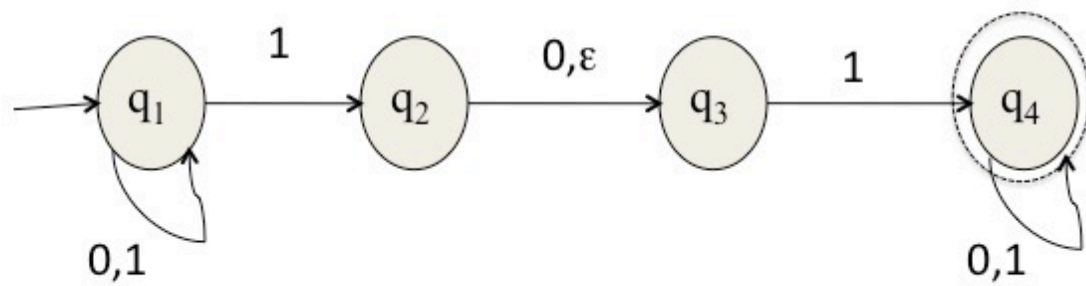
Equivalenza tra DFA e NFA

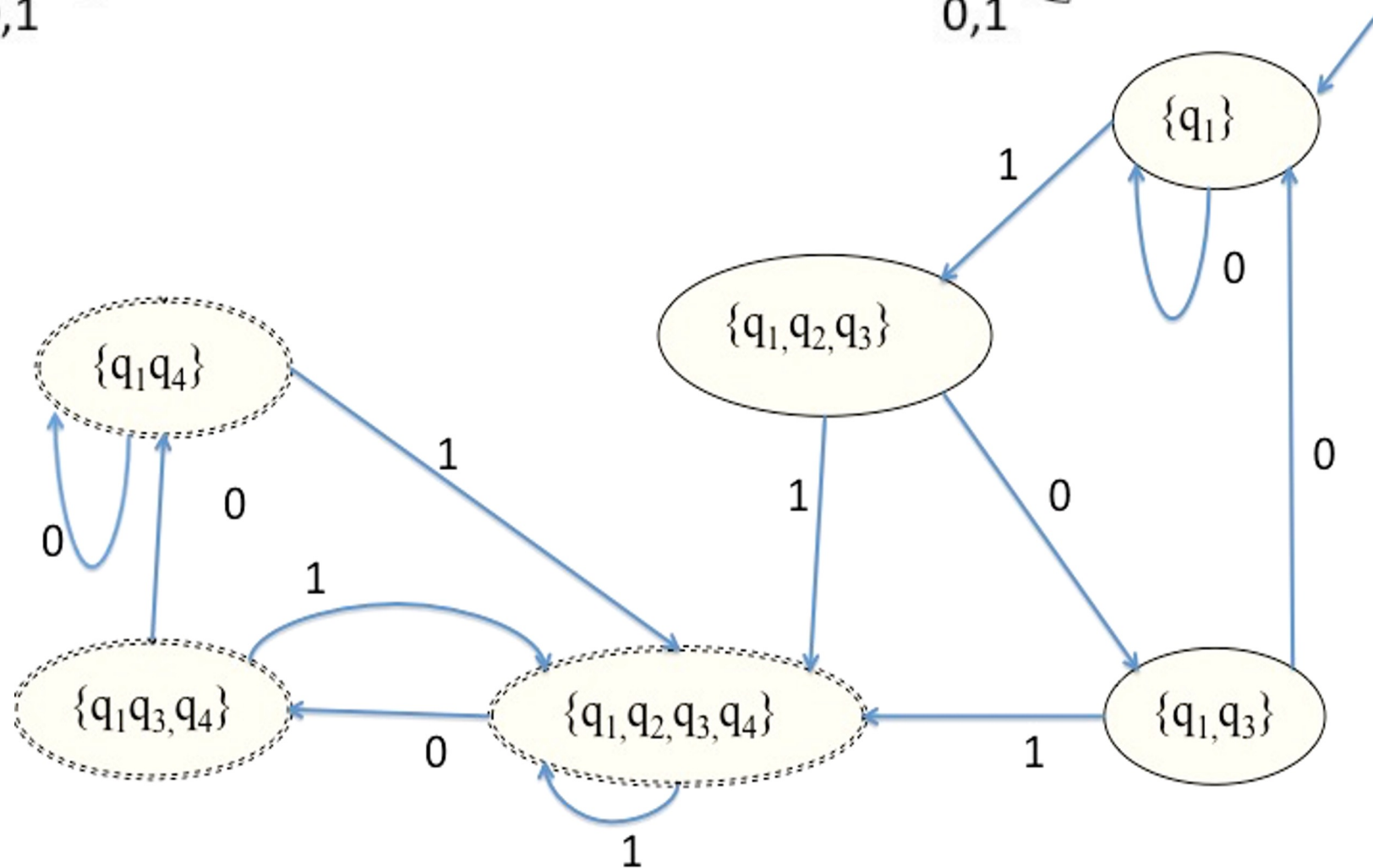
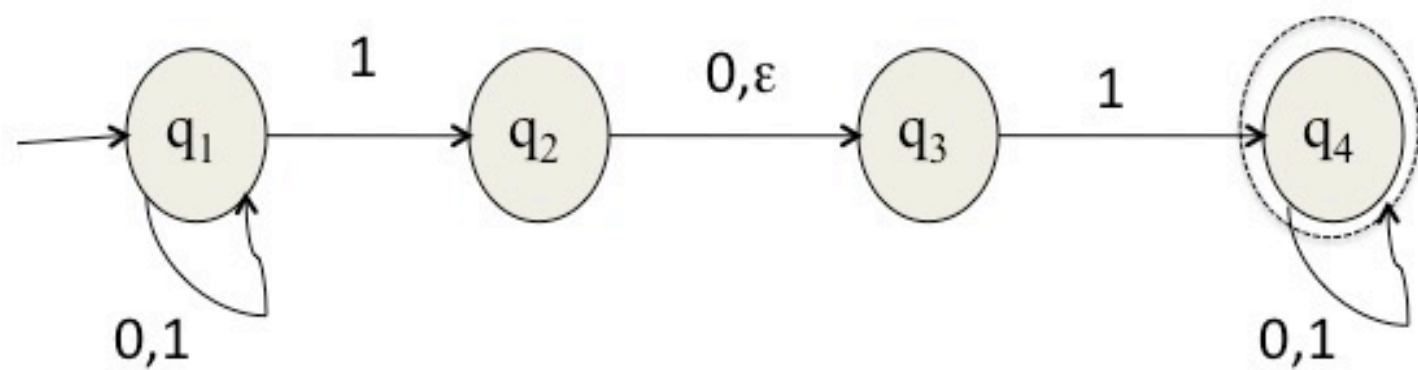
Def.: due macchine sono **equivalenti** se riconoscono lo stesso linguaggio.

Teorema

Ogni NFA N ha un DFA M equivalente;
cioè, se N è un NFA, allora esiste DFA M t.c. $L(M) = L(N)$.

Dim. Costruiamo a partire da N , il DFA M equivalente





Sia $L = L(N)$ per NFA $N = (Q_N, \Sigma, f_N, q_N, F_N)$

Costruiamo DFA $D = (Q_D, \Sigma, f_D, q_D, F_D)$.

Partiamo da D' che non considera le e-transition:

$$Q = P(Q_N)$$

$$f(R, a) = \bigcup_{r \in R} f_N(r, a),$$

per ogni $R \in Q$ e $a \in \Sigma$,

$$q = \{q_N\}$$

$$F = \{R \in Q \mid R \cap F_N \neq \emptyset\}$$

- Per ogni stato in f_N , se vi è ε -transition allora dobbiamo considerarla
- Definiamo
 $E(R) = R \cup \{q \mid q \text{ raggiungibile da uno stato in } R \text{ con 1 o più archi con label } \varepsilon\}$

$$Q_D = P(Q_N)$$

$$f_D(R, a) = \bigcup_{r \in R} E(f_N(r, a)),$$

per ogni $R \in Q_D$ e $a \in \Sigma$,

$$q_D = E(\{q_N\})$$

$$F_D = \{R \in Q_D \mid R \cap F_N \neq \emptyset\}$$

- Risulta, per ogni $x \in \Sigma^*$,

$$f_D^*(q_D, x) = f_N^*(q_N, x)$$

Si può provare per induzione

- D simula N su ogni input x
- D accetta sse N accetta
- $L = L(N) = L(D)$

Nota $f_N^*(q_N, x) =$ insieme stati raggiungibili da q_N su input x

$$f_N^*(q_N, \varepsilon) = E(\{q_N\})$$

$$f_N^*(q_N, xa) = \bigcup_{r \in f_N^*(q_N, x)} E(f_N(r, a))$$

Mostriamo per induzione su $|w|$ che

$$f^*_D(\{q_0\}, w) = f^*_N(q_0, w) \quad [\text{poniamo } q_0 = q_N]$$

Base: $w = \epsilon$. L'enunciato segue dalla definizione.

Mostriamo per induzione su $|w|$ che

$$f^*_D(\{q_0\}, w) = f^*_N(q_0, w) \quad [\text{poniamo } q_0 = q_N]$$

Base: $w = \epsilon$. L'enunciato segue dalla definizione.

Passo:

$$\begin{aligned} f^*_D(\{q_0\}, xa) &= f_D(f^*_D(\{q_0\}, x), a) \quad (\text{def.}) \\ &= f_D(f^*_N(q_0, x), a) \quad (\text{i.i.}) \\ &= \bigcup_{r \in f^*_N(q_0, x)} E(f_N(r, a)) \quad (\text{def.}) \\ &= f^*_N(q_0, xa) \end{aligned}$$

Linguaggio riconosciuto da NFA $\Leftrightarrow$ Regolare

Corollario

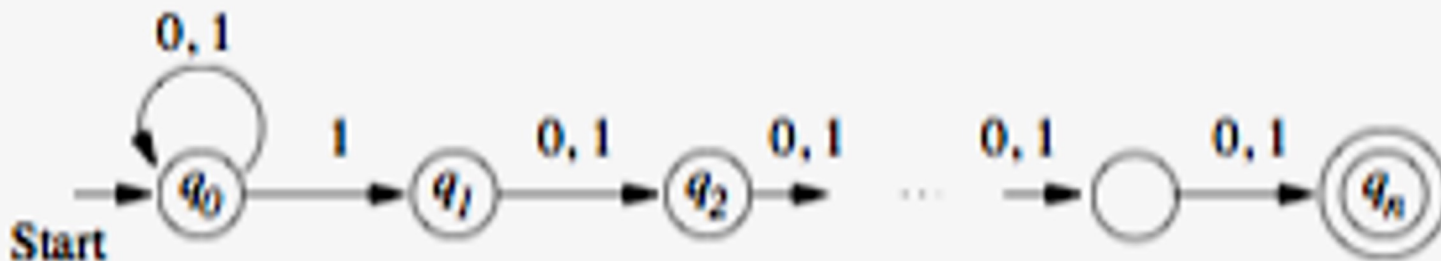
Linguaggio L è regolare sse esiste NFA che riconosce A .

Dim.

Se L è regolare, allora esiste DFA
ma ogni DFA è anche un NFA, quindi esiste NFA per L .

Da Teorema precedente, ogni NFA ha equivalente DFA.
Quindi se esiste NFA allora esiste DFA per L

**Esiste un NFA N con $n + 1$ stati che non ha nessun DFA
equivalente con meno di 2^n stati**



$$L(N) = \{x1c_2c_3 \cdots c_n : x \in \{0,1\}^*, c_i \in \{0,1\}\}$$

Idea:

Supponiamo esista DFA D equivalente con meno di 2^n stati.

D deve ricordare gli ultimi n simboli che ha letto.

Ci sono 2^n sequenze di bit $a_1a_2 \cdots a_n$ da *ricordare*

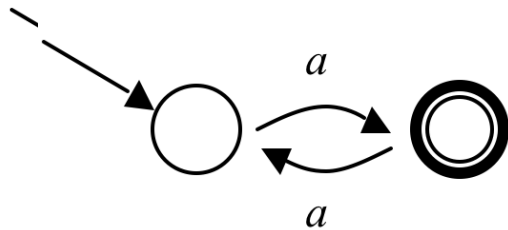
CONCATENAZIONE DI LINGUAGGI REGOLARI

$$AB = \{ vw \mid v \in A, w \in B \}.$$

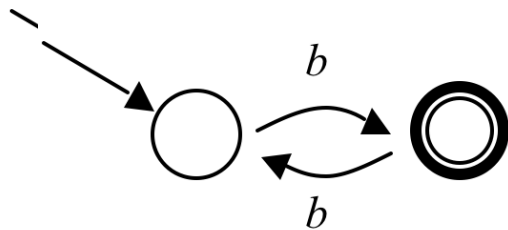
Teorema

La classe dei linguaggi regolari è chiusa per l'operazione di concatenazione.

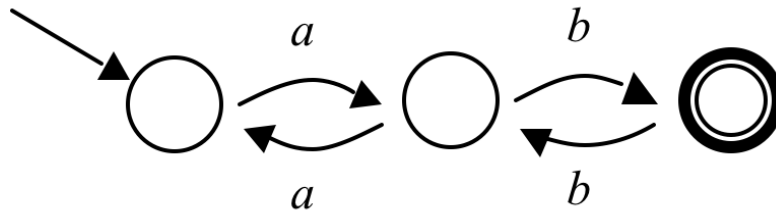
Concatenazione ?



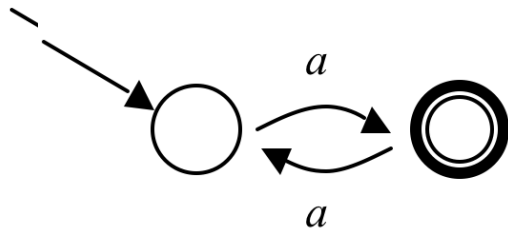
$\{ a^n \mid n \text{ dispari} \}.$



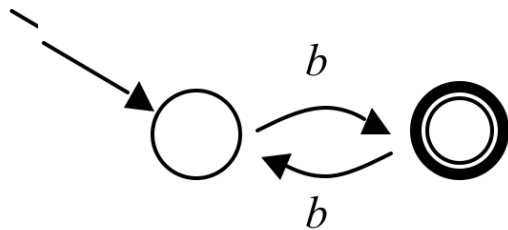
$\{ b^n \mid n \text{ dispari} \}.$



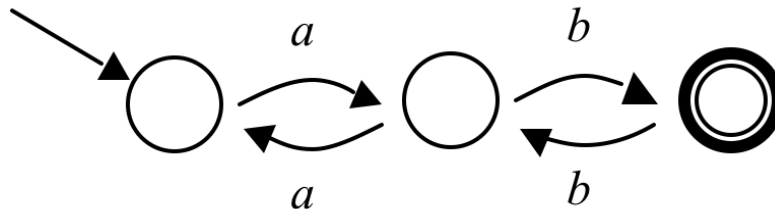
Concatenazione **scorretta**



$\{ a^n \mid n \text{ dispari} \}.$

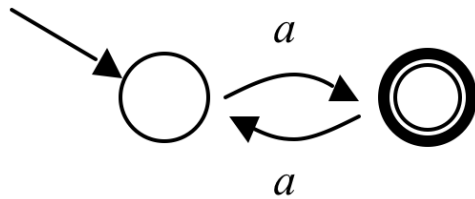


$\{ b^n \mid n \text{ dispari} \}.$

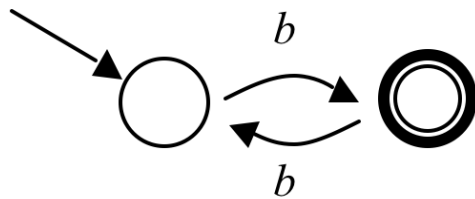


Riconosce abbaab!!

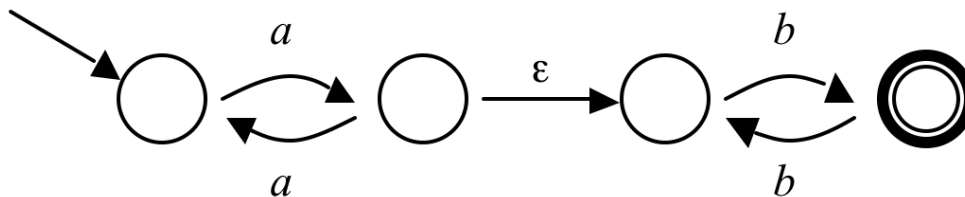
Concatenazione Corretta



$\{ a^n \mid n \text{ dispari} \}.$

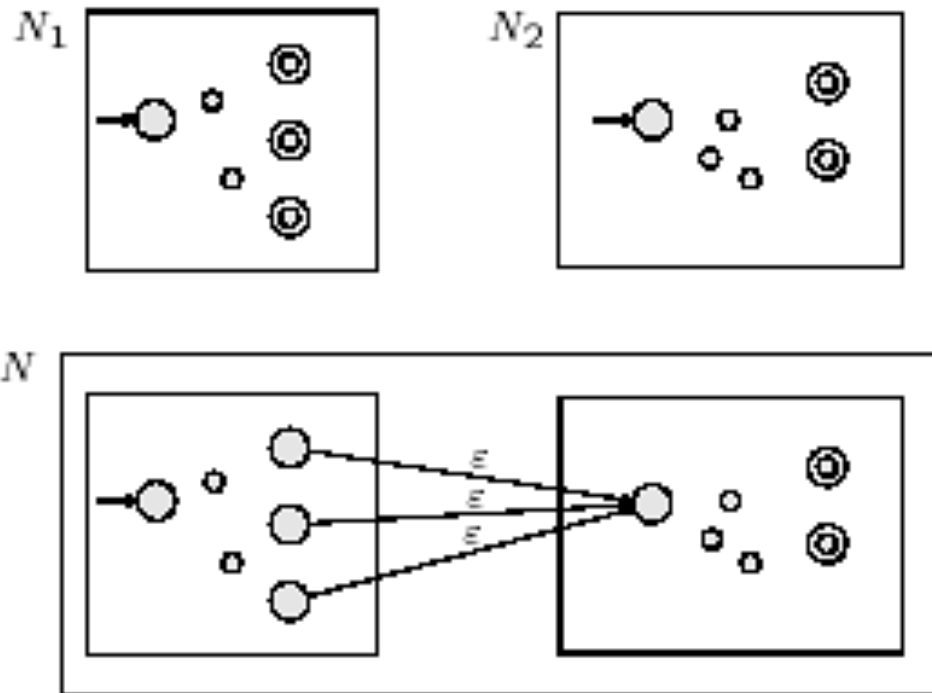


$\{ b^n \mid n \text{ dispari} \}.$

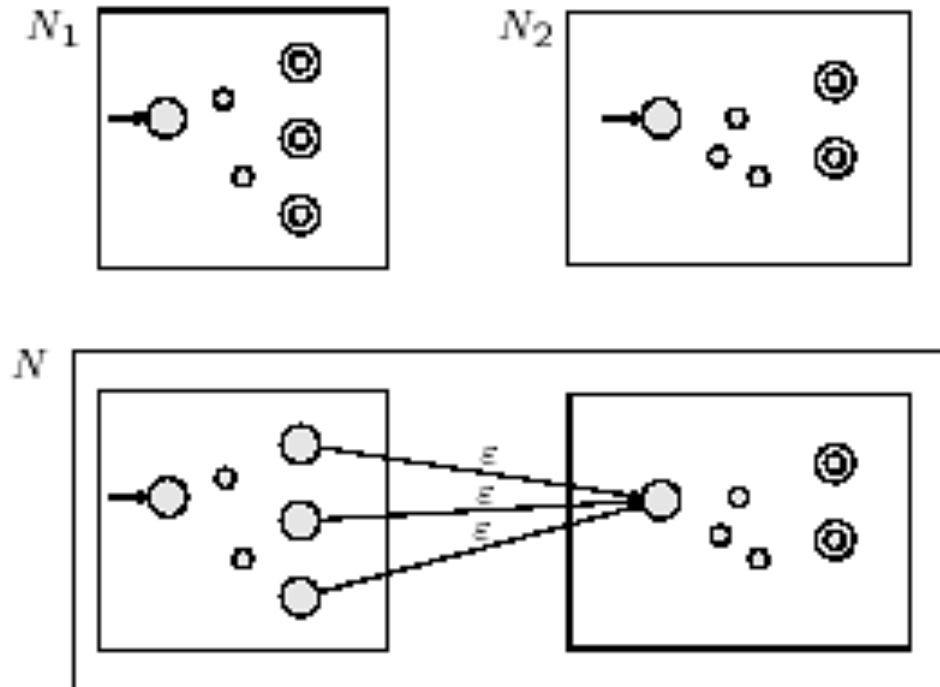


$\{ xy \mid x \in A \text{ e } y \in B \}$

Dim Idea: NFA N per $L_1 L_2$:



Dim Idea: NFA N per $L_1 L_2$:



$w=uv$

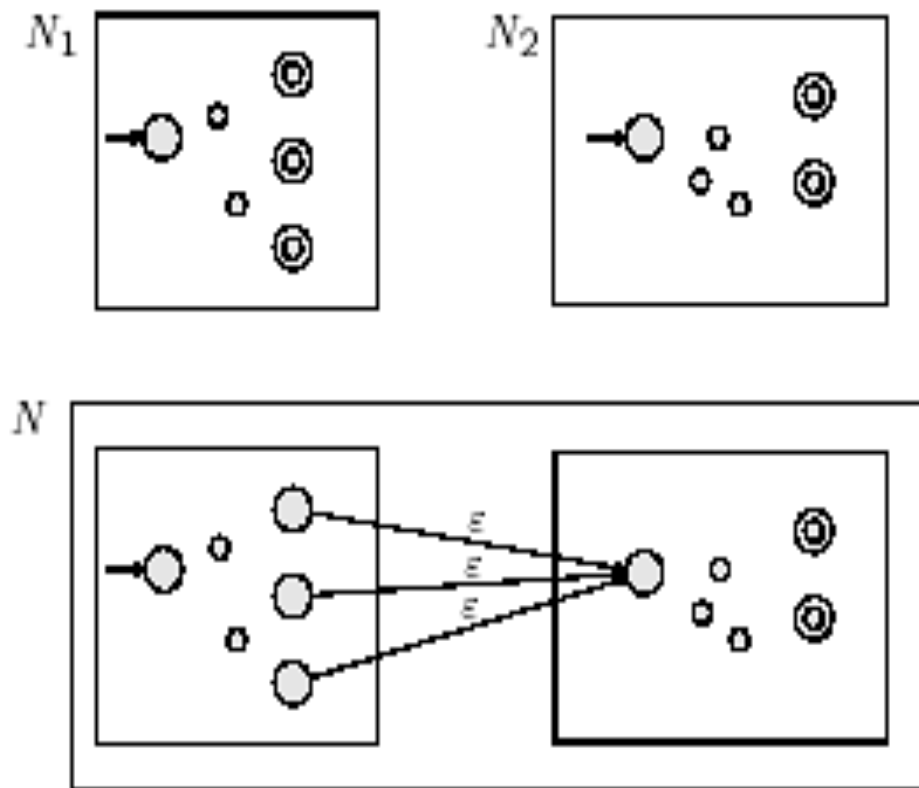
u in L_1

AND

v in L_2

Quindi esiste una
computazione accettante che
usa ϵ -transition dopo u

Dim Idea: NFA N per $L_1 L_2$:



w non in $L_1 L_2$

Comunque scriviamo

$w=uv$:

Se supponiamo che la ϵ -transition
dal primo al secondo automa

si ha dopo u

Sappiamo che

u non in L_1

OR

v non in L_2

Quindi non possiamo avere
computazione accettante

L_1 linguaggio riconosciuto da NFA $N_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$.

L_2 linguaggio riconosciuto da NFA $N_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$.

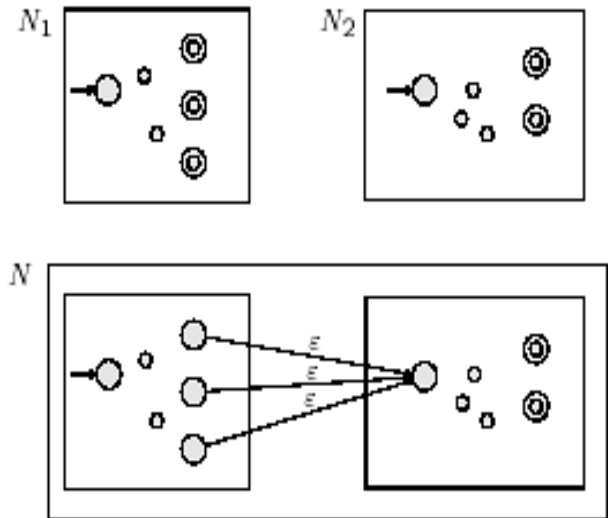
NFA $N = (Q, \Sigma, \delta, q_1, F_2)$ per $L_1 L_2$:

$$Q = Q_1 \cup Q_2$$

Stato start q_1 , stesso di N_1 .

Stati finali F_2 , stessi di N_2 .

Funzione di transizione:



$$\delta(q, a) = \begin{cases} \delta_1(q, a) & \text{if } q \in Q_1 - F_1, \\ \delta_1(q, a) & \text{if } q \in F_1 \text{ and } a \neq \epsilon, \\ \delta_1(q, a) \cup \{q_2\} & \text{if } q \in F_1 \text{ and } a = \epsilon, \\ \delta_2(q, a) & \text{if } q \in Q_2. \end{cases}$$

Esempio: vogliamo applicare il metodo visto per costruire un NFA N che riconosce il linguaggio $L(N)$ formato da tutte le stringhe su alfabeto $\{0,1\}$ che iniziano con 00 e terminano con 1

Una computazione accettante quando in input si ha 001011

0010 non è accettata

Si può estendere all'operazione di Kleene star:

$$L^* = \{ x_1 x_2 \cdots x_k \mid k \geq 0, x_i \text{ in } L \}.$$

Teorema

La classe dei linguaggi regolari è chiusa per l'operazione Kleene-star.

DIM IDEA. Se N_1 riconosce L . Costruiamo N da N_1 in cui ogni stato finale è collegato da ϵ -transizione allo stato iniziale

N_1



N

